

Problem 1

- Part A

lang.rkt:

```
(expression ("minus(" expression ")") minus-exp)
```

interp.rkt:

```
(minus-exp (exp1)
  (let ((val1 (value-of exp1 env)))
    (let ((num1 (expval->num val1)))
      (num-val
        (- 0 num1))))))
```

- Part B

lang.rkt:

```
(expression ("prime?(" expression ")") prime-exp)
```

interp.rkt:

```
(prime-exp (exp1)
  (let ((val1 (value-of exp1 env)))
    (let ((num1 (expval->num val1)))
      (bool-val (prime? num1 2))))))
```

```
(define (prime? num div) (
  cond ((= 1 num) #f)
        ((> (* div div) num) #t)
        ((= (remainder num div) 0) #f)
        (else (prime? num (+ 1 div)))))
```

Problem 2

Let p0 = [x = 20, y = 15, z = 10]

```
(value-of
  <<let n = 15
    in if zero?(-(x, n)) then
      -(x, y)
      else -(-(z, n), -(y, x))>>
```

p0)

```
= (value-of
  <<if zero?(-(x, n)) then
    -(x, y)
    else -(-(z, n), -(y, x))>>
```

(n=[15] p0) [1])

Let p1 = [x = 20, y = 15, z = 10, n = 15] [2]

```

= (if (expval->bool (value-of <<zero?(-(x, n))>> ρ1))
      (value-of <<-(x, y)>> ρ1)
      (value-of <<-( -(z, n), -(y, x))>> ρ1))

= (if #f [3]
      (value-of <<-(x, y)>> ρ1)
      (value-of <<-( -(z, n), -(y, x))>> ρ1))

= (value-of <<-( -(z, n), -(y, x))>> [4]>> ρ1)

= [(-
  [(value-of <<-(z, n) [5]>> ρ1) ]
  [(value-of <<-(y, x) [6]>> ρ1) ])]

= 0 [7]

```